

Planning to Explore via Self-Supervised World Models

Ramanan Sekar, Oleh Rybkin, Kostas Daniilidis, Pieter Abbeel, Danijar Hafner, Deepak Pathak

Presenter: Hongming Zhang

Date: 2022.06.10

Introduction



- ICML 2020

Plan2Explore: **self-supervised exploration** and **fast adaptation** to new tasks.

During exploration, our agent acts efficiently by leveraging **planning** to seek out expected future novelty (to learn world models).

After exploration, the agent quickly adapts to multiple downstream tasks in a **zero or a few-shot manner** (using world models).

Motivation

Reinforcement learning tends to be task specific. How can an agent quickly **generalize** to unseen tasks it has never experienced before in **a zero or few-shot manner**?

- Explore: **not keep collecting data for each new task**. We explore the environment once **without reward**.
- Adaptation: the agent is provided with **downstream reward** functions to solve the tasks with limited or no further environment interaction.

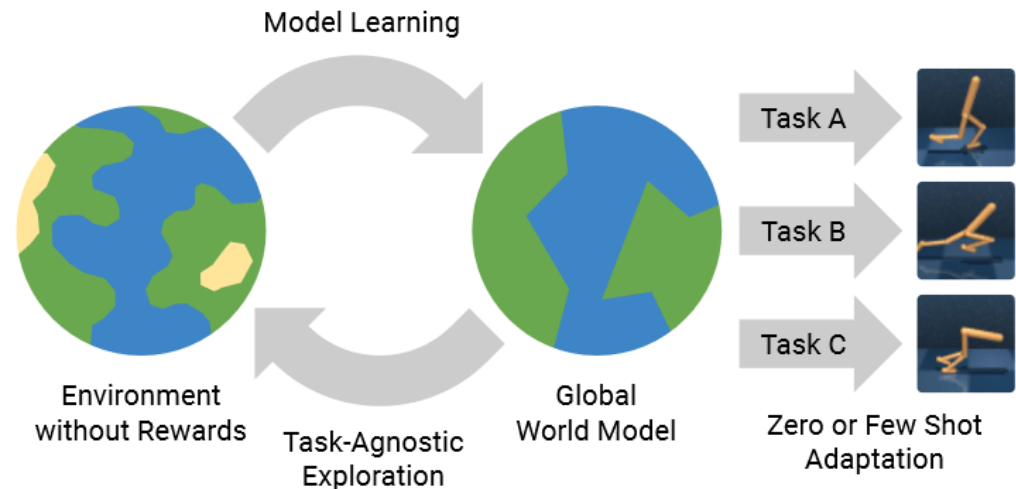


Figure 1. The agent first leverages planning to explore in a self-supervised manner, without task-specific rewards, to efficiently learn a global world model. After the exploration phase, it receives reward functions at test time to adapt to multiple downstream tasks, such as standing, walking, running, and flipping using either zero or few tasks-specific interactions.

Algorithm



- World models: PlaNet (RSSM)
- Explore: Dreamer
- Adaptation: Zero and Few-Shot Task Adaptation

Algorithm



- World models

Image encoder: $h_t = e_\theta(o_t)$

Posterior dynamics: $q_\theta(s_t|s_{t-1}, a_{t-1}, h_t)$

Prior dynamics: $p_\theta(s_t|s_{t-1}, a_{t-1})$

Reward dynamics: $p_\theta(r_t|s_t)$

Image decoder: $p_\theta(o_t|s_t)$

- The image encoder is implemented as a CNN, and the posterior and prior dynamics share an **RSSM**.
- The reward predictor and image decoder **provide a rich learning signal** to the dynamics.
- The distributions are parameterized as **diagonal Gaussians**.

Algorithm

- World models - Recurrent state-space model (RSSM)

Deterministic state model: $h_t = f(h_{t-1}, s_{t-1}, a_{t-1})$

Stochastic state model: $s_t \sim p(s_t|h_t)$

Observation model: $o_t \sim p(o_t|h_t, s_t)$

Reward model: $r_t \sim p(r_t|h_t, s_t)$

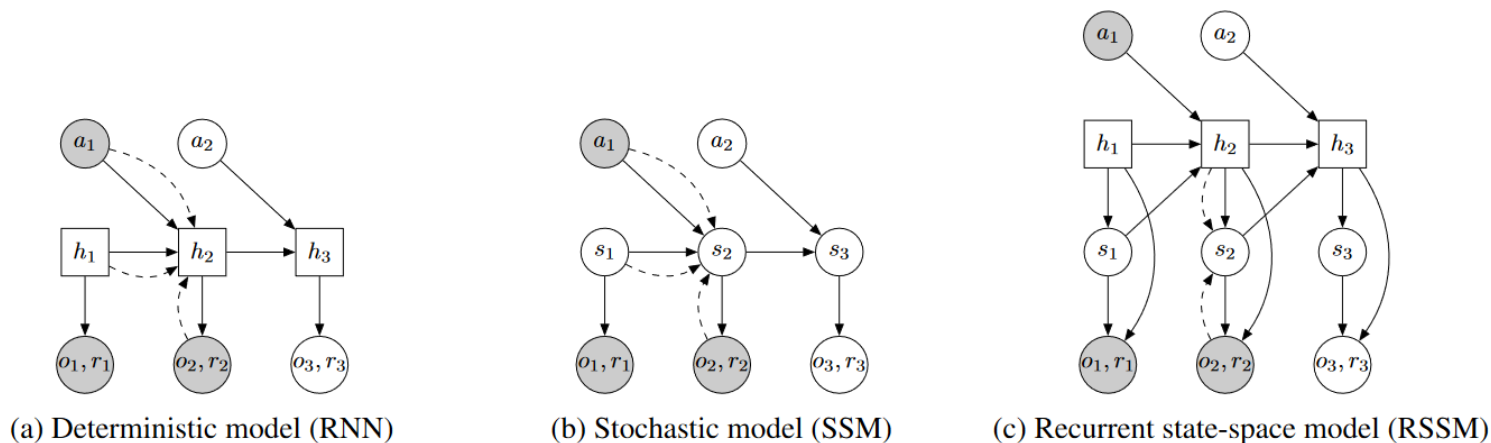


Figure 2: Latent dynamics model designs. In this example, the model observes the first two time steps and predicts the third. **Circles represent stochastic variables and squares deterministic variables.** Solid lines denote the generative process and dashed lines the inference model. (a) Transitions in a recurrent neural network are purely deterministic. This prevents the model from capturing multiple futures and makes it easy for the planner to exploit inaccuracies. (b) Transitions in a state-space model are purely stochastic. This makes it difficult to remember information over multiple time steps. (c) We split the state into stochastic and deterministic parts, allowing the model to robustly learn to predict multiple futures.

Algorithm

- Explore - overview

Train an exploration policy inside of the world model to **seek out novel states**. Novelty is estimated by **ensemble disagreement in latent predictions** made by 1-step transition models trained alongside the global recurrent world model.

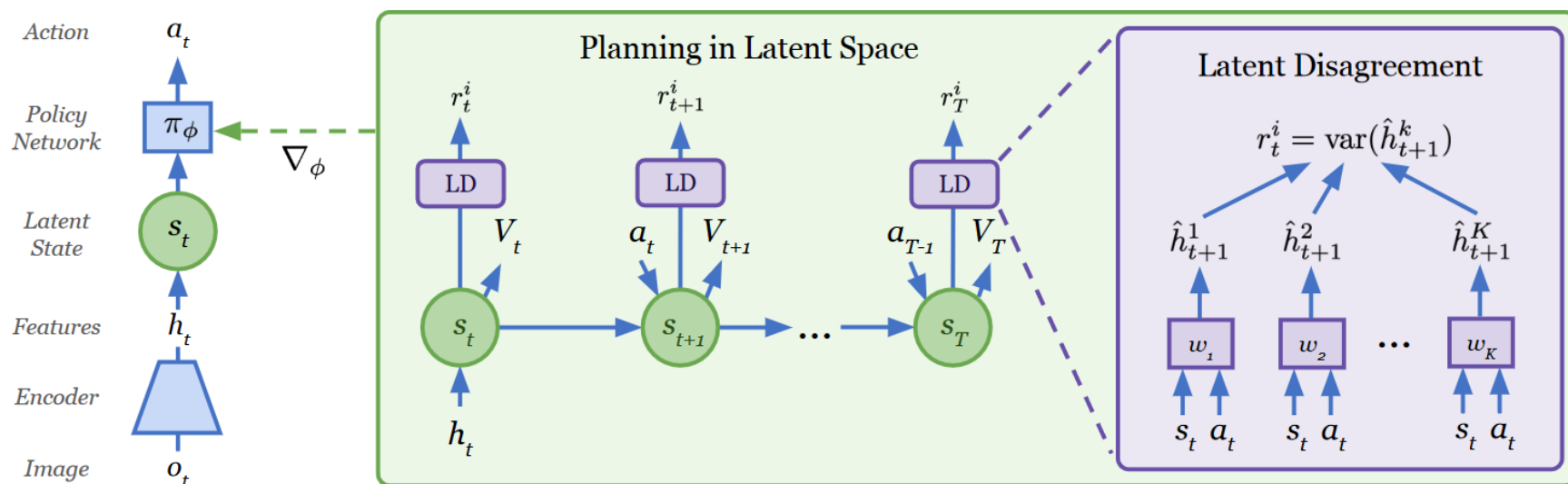


Figure 2. Overview of Plan2Explore. Each observation o_t at time t is first encoded into features h_t which are then used to infer a recurrent latent state s_t . At each training step, the agent leverages planning to explore by imagining the consequences of the actions of policy π_ϕ using the current world model. The planning objective is to maximize expected novelty r_t^i over all future time steps, computed as the **disagreement in the predicted next image embedding h_{t+1} from an ensemble of learned transition dynamics w_k** . This planning objective is backpropagated all the way through the imagined rollout states to improve the exploration policy π_ϕ . The learned model is used for planning to explore in latent space, and the data collected during exploration is in turn used to improve the model. This world model is then later used to plan for novel tasks at test time by replacing novelty reward with task reward.

Algorithm



- Explore – reward (latent disagreement)

Define a bootstrap **ensemble of one-step predictive models**

$$q(h_{t+1}|\omega_k, s_t, a_t) = N(\mu(\omega_k, s_t, a_t), 1), k \in [1, K]$$

Quantify model uncertainty as the **variance** over predicted **means** of the different ensemble members and use this disagreement as the intrinsic reward

$$\text{ir}_t = D(s_t, a_t) = \text{Var}(\mu(\omega_k, s_t, a_t)), k \in [1, K]$$

The disagreement is **positive** for novel states.

It eventually **reduces to zero** even for stochastic environments because all one-step predictions converge to the mean of the next input.

The intrinsic reward is **non-stationary**.

Algorithm

- Explore - learn the exploration policy using **Dreamer**

Learn two neural networks that operate on latent states of the model:

Actor: $\pi(a_t|s_t)$

Value: $V(s_t)$

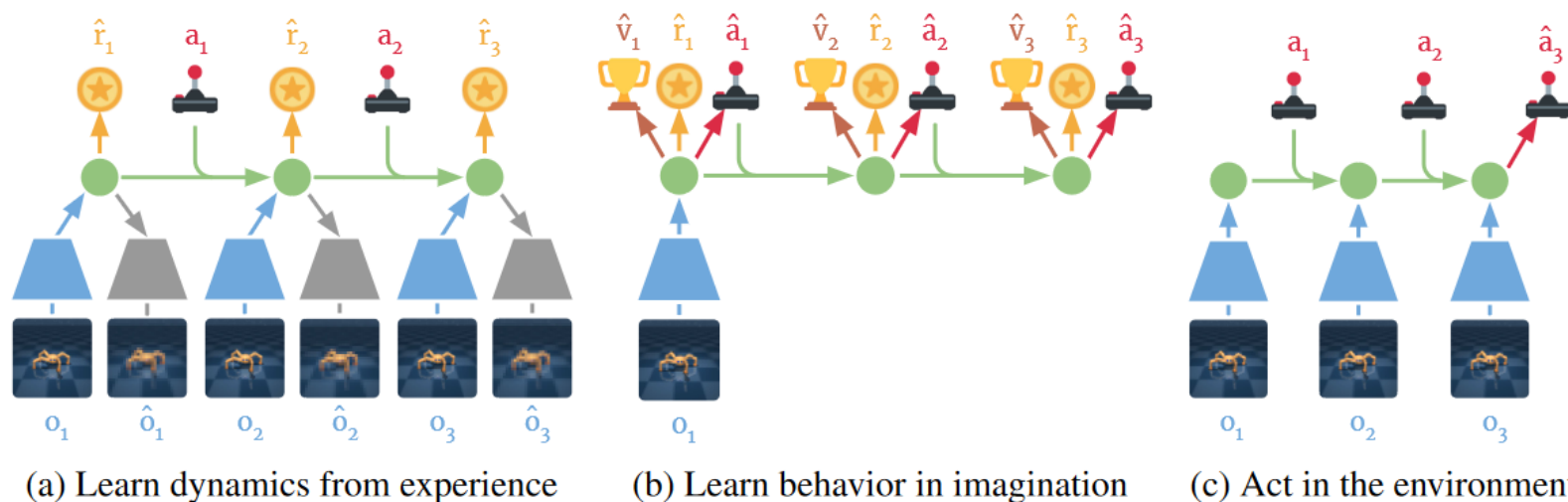


Figure 3: Components of Dreamer. (a) From the dataset of past experience, the agent learns to encode observations and actions into **compact latent states** (●), for example via **reconstruction**, and predicts environment **rewards** (★). (b) In the compact latent space, Dreamer predicts **state values** (🏆) and **actions** (♟) that maximize future value predictions by propagating gradients back through imagined trajectories. (c) The agent encodes the history of the episode to compute the current model state and predict the next action to execute in the environment. See [Algorithm 1](#) for pseudo code of the agent.

Algorithm



- Explore - learn the exploration policy using **Dreamer**

The action model implements the policy and aims to predict actions that solve the **imagination environment**:

$$a_\tau \sim q_\phi(a_\tau | s_\tau)$$

The value model estimates the expected **imagined rewards** that the action model achieves

$$v_\psi(s_\tau) \approx E_{q(\cdot | s_\tau)}(\sum_{\tau=t}^{t+H} \gamma^{\tau-t} r_\tau)$$

The action model **uses analytic gradients through the learned dynamics** to maximize the value estimates. This is similar to deterministic or reparameterized actor critics such as DDPG and SAC.

Algorithm 1 Planning to Explore via Latent Disagreement

- 1: **initialize:** Dataset D from a few random episodes.
 - 2: World model M.
 - 3: Latent disagreement ensemble E.
 - 4: Exploration actor-critic π_{LD} .
 - 5: **while** exploring **do**
 - 6: Train M on D.
 - 7: Train E on D.
 - 8: Train π_{LD} on LD reward in imagination of M.
 - 9: Execute π_{LD} in the environment to expand D.
 - 10: **end while**
 - 11: **return** Task-agnostic D and M.
-

Algorithm



- Explore - benefits
 - Retrospective novelty

Model-free exploration methods usually first act in the environment, the novelty of inputs is computed **after the agent has already reached** them.

Hence, it seeks out previously novel inputs that have already been visited and would not be novel anymore.

- Planning to explore

One should directly seek out future inputs that are expected to be novel. Instead of maximizing intrinsic rewards in retrospect, we learn a world model to **plan ahead** and seek out the expected novelty of future situations.

The exploration policy is **optimized purely from trajectories imagined** under the model to maximize the intrinsic rewards computed by the model itself.

- Adaptation - Zero and Few-Shot Task Adaptation

Optimize a task policy by **imagination inside of the world model**.

After exploration, the agent is given a downstream task in the form of a reward function that it should adapt to with no or limited additional environment interaction.

Algorithm 2 Zero and Few-Shot Task Adaptation

- 1: **input:** World model M .
 - 2: Dataset D without rewards.
 - 3: Reward function R .
 - 4: **initialize:** Latent-space reward predictor $\hat{R}$.
 - 5: Task actor-critic π_R .
 - 6: **while** adapting **do**
 - 7: Distill R into $\hat{R}$ for sequences in D .
 - 8: Train π_R on $\hat{R}$ in imagination of M .
 - 9: Execute π_R for the task and report performance.
 - 10: Optionally, add task-specific episode to D and repeat.
 - 11: **end while**
 - 12: **return** Task actor-critic π_R .
-

Experiments

- Few-Shot Adaptation

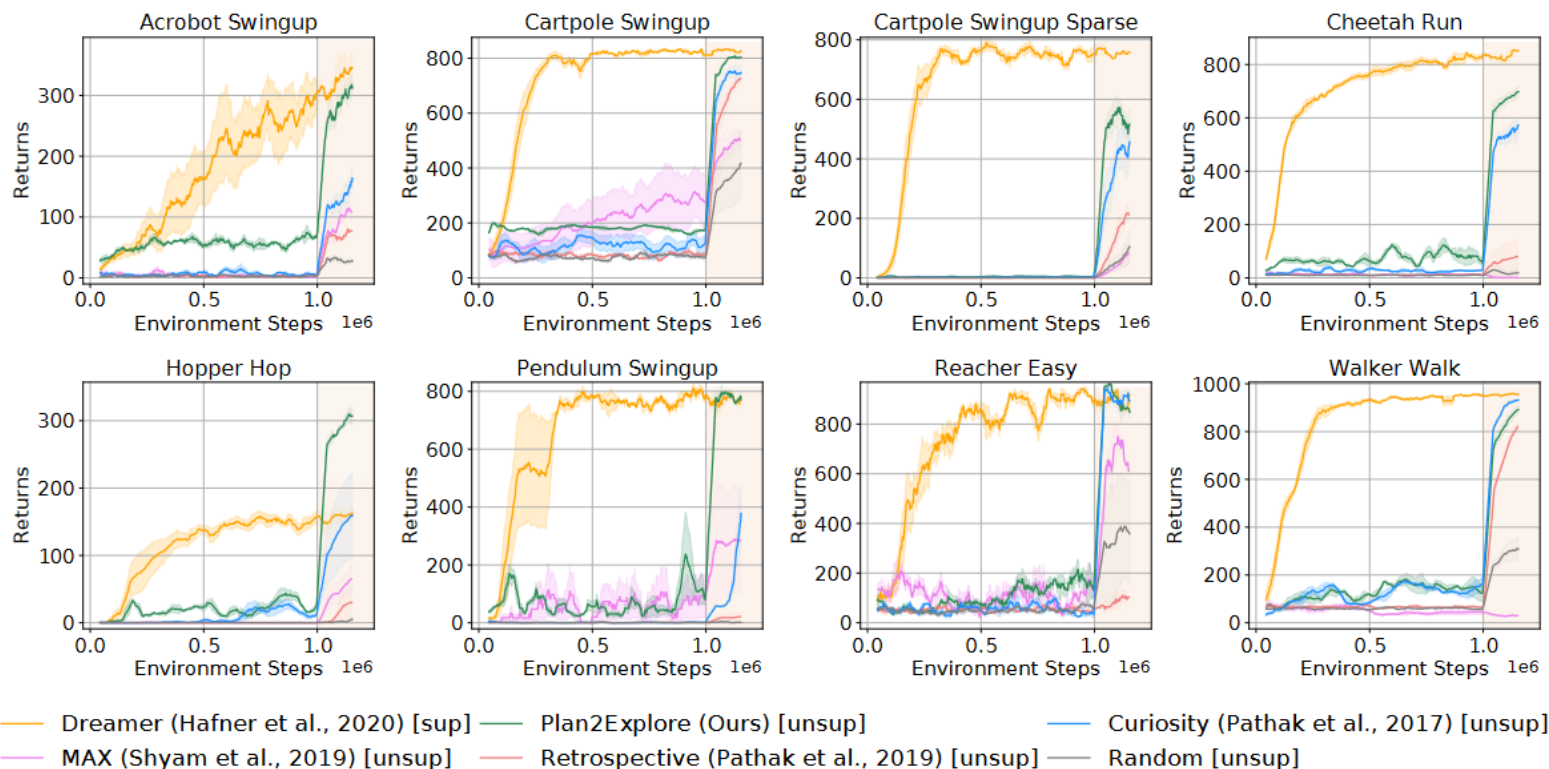


Figure 4. Performance on few-shot adaptation from raw pixels without state-space input. After the exploration phase of 1M steps (white background), during which the agent does not observe the reward and thus does not solve the task, we let the agent collect a small amount of data from the environment (shaded background). We see that Plan2Explore is able to explore the environment efficiently in only 1000 episodes, and then adapt its behavior immediately after observing the reward. Plan2Explore adapts rapidly, producing effective behavior competitive to state-of-the-art supervised reinforcement learning in just a few collected episodes.

Experiments

- Zero-Shot Adaptation

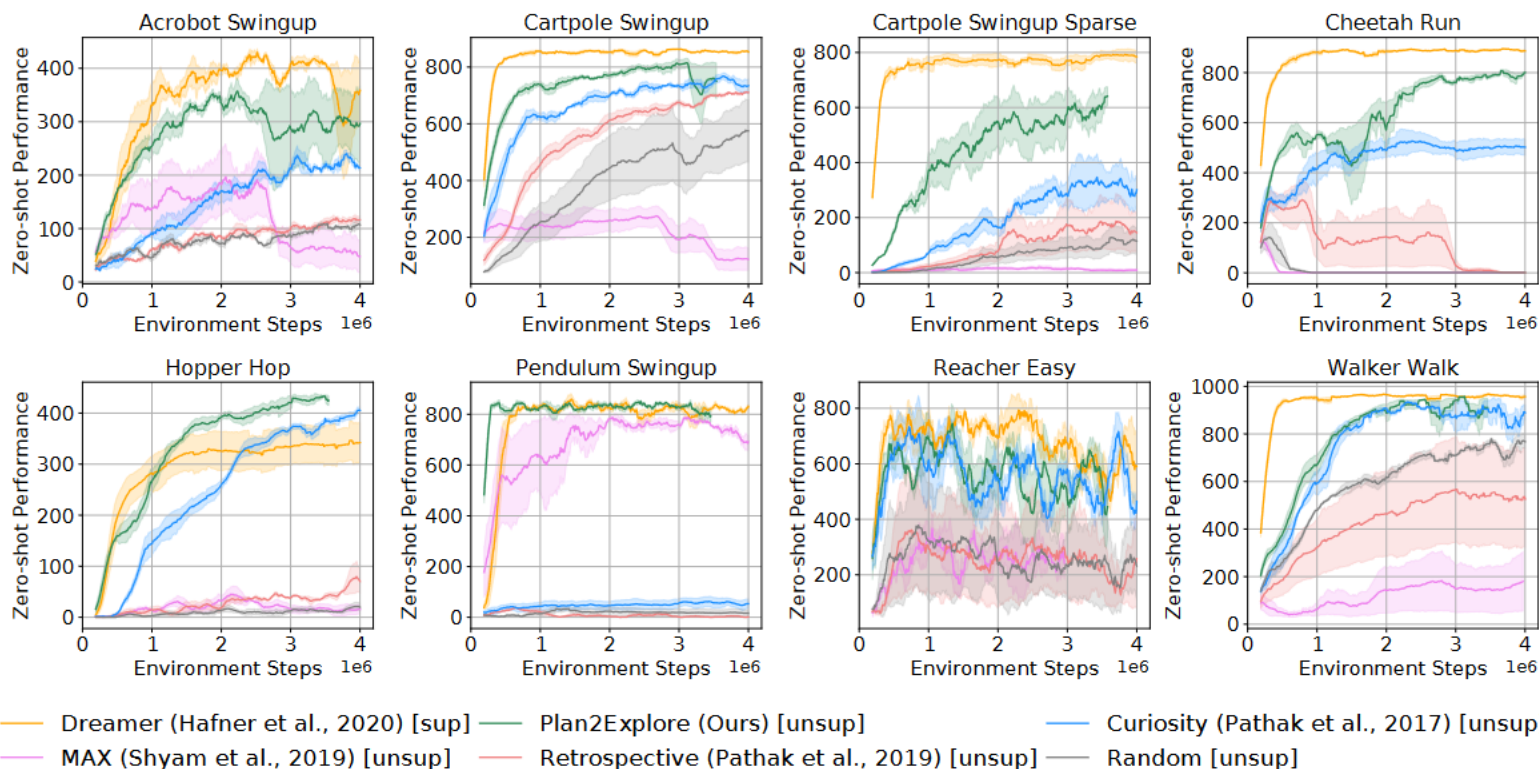


Figure 3. Zero-shot RL performance from raw pixels. After training the agent without rewards, we provide it with a task by specifying the reward function at test time. Throughout the exploration, we take snapshots of the agent to train a task policy on the final task and plot its zero-shot performance. We see that Plan2Explore achieves state-of-the-art zero-shot task performance on a range of tasks, and even demonstrates competitive performance to Dreamer (Hafner et al., 2020), a state-of-the-art supervised RL agent. This indicates that Plan2Explore is able to explore and learn a global model of the environment that is useful for adapting to new tasks, demonstrating the potential of self-supervised RL. Results on all 20 tasks are in the appendix Figure 1 and videos on the project website.

Experiments

- Generalization

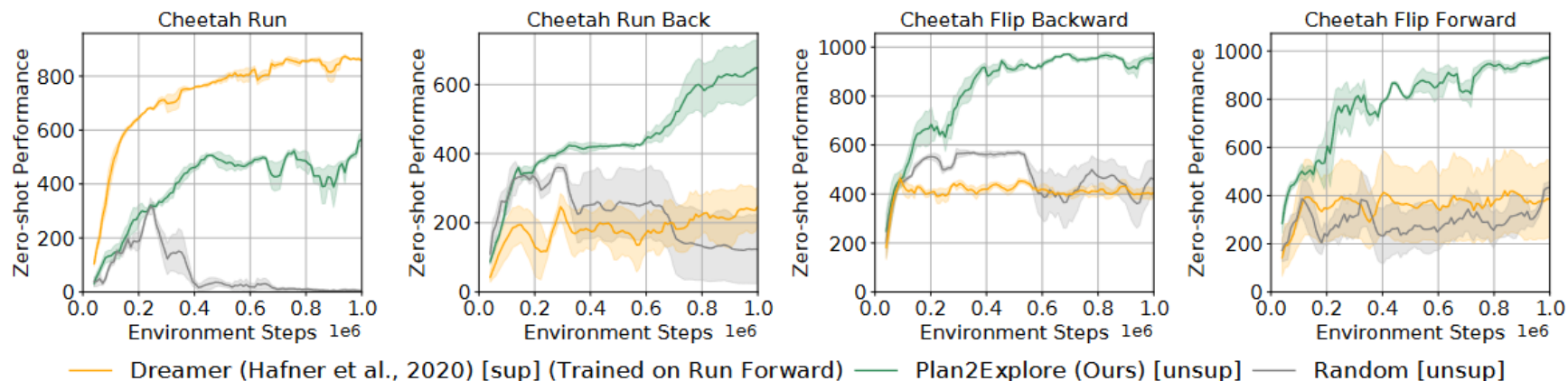


Figure 5. Do task-specific models generalize? We test Plan2Explore on zero-shot performance on four different tasks in the cheetah environment from raw pixels without state-space input. Throughout the exploration, we take snapshots of policy to plot its zero-shot performance. In addition to random exploration, we compare to an oracle agent, Dreamer, that uses the data collected when trained on the run forward task with rewards. Although Dreamer trained on 'run forward' is able to solve the task it is trained on, it struggles on the other tasks, indicating that it has not learned a global world model.

Conclusion



Plan2Explore, a self-supervised reinforcement learning agent that leverages planning to **efficiently explore** visual environments without rewards.

After the exploration phase, the learned world model is used to train downstream task policies in imagination via offline reinforcement learning, **without any further environment interaction**.



THANKS